



Non-Invasive Blood Glucose Level Estimation Using Smartphone PPG Signals

Authors

Abdelrahman Rabie

Mostafa Magdy

Sherif Nasser

Supervisors

Prof. Osama Omar

Dr. Mohammed Abdelnasser

Dr. Ahmed Nor

Eng. Abdelrahman Yahia

A Graduation Project Thesis is submitted to Electrical Engineering Dep,
Faculty of Engineering, Aswan University in partial fulfillment requirements
for the degree of B.Sc. in Electrical Department
(Computer and Systems Section)

July 2026, Aswan, Egypt

Acknowledgment

All praise is due to ALLAH, our only Lord and Ruler. If it weren't for ALLAH's help and mercy, we wouldn't have been able to make it this far into the project.

We would like to thank our supervisor, Prof. Mountasser Mohamed, for supporting our graduation project.

Also, a special thanks to our supervisor and mentor, Dr. Ahmed Mostafa, for his guidance and help throughout the year, and for providing us with the needed advice and components.

Thanks to Eng. Hossam Mady for believing in us and his continuous support, which helped us achieve this project.

We couldn't forget Eng. Dyaa Elshater, who was always prompt in helping us with everything we needed from him.

Finally, we would like to express our gratitude to our parents and close friends. Without their understanding and encouragement over the past few years, it would have been impossible for us to complete our studies.

Abstract

This project aims to design and develop a smart wheelchair with advanced control mechanisms to enhance mobility and independence for individuals with physical disabilities.

The smart wheelchair integrates multiple advanced control mechanisms, including joystick operation, face recognition, and eye tracking technology, to accommodate users with varying levels of mobility and dexterity.

The joystick control provides a familiar and intuitive interface for users who retain upper body mobility. For users with limited hand functionality, the face recognition control mode allows for navigation through facial movements and expressions. Furthermore, the eye tracking system offers an innovative solution for users with severe physical limitations, enabling wheelchair control through eye movements. By leveraging state-of-the-art technologies in computer vision and human-computer interaction, the smart wheelchair ensures a versatile and user-friendly interface.

Additionally, the wheelchair is integrated with a Healthcare System that monitors and records sensor measurements related to the user's health. This data can be used for ongoing health assessments and to provide real-time feedback to healthcare providers.

The goal of this project is to create an adaptable, reliable, and safe mobility solution that significantly improves the quality of life for individuals with mobility challenges.

List of Figures

3.1	The New Record Form interface where patient demographics and Accu-Chek readings are entered.	11
3.2	The user interfaces designed for checking the finger alignment and executing the 20-second signal capture sequence.	13
3.3	Data management screens showing local storage monitoring and the final data export engine workflow.	14

Contents

Cover Page	1
Acknowledgment	1
Abstract	2
List of Figures	3
Table of Contents	3
1 Introduction	5
2 Related Work	6
3 Methodology	7
3.1 Introduction	7

3.2	Phase 1: Data Acquisition	7
3.2.1	The Data Collection Campaign at University Hospital	8
3.2.2	Our Android Application: DiabetesCollector	9
3.2.3	Patient Profiles and Ground Truth Entry	9
3.2.4	Handling Data without Overlaps (UUID Strategy)	12
3.2.5	The Signal Recording Process (The 20-Second Rule)	12
3.2.6	Exporting and Merging the Final Dataset	13
3.3	Phase 2: Data Preprocessing and Structuring	16
3.3.1	Video Orientation Normalization	16
3.3.2	Hierarchical Dataset Re-organization	17
3.3.3	Quality Control and Data Filtering	19
3.3.4	Automation Python Script for Restructuring	19
3.4	Phase 3: rPPG Extraction	20
3.4.1	Raw Temporal RGB Extraction	21
3.4.2	Frame Rate Normalization Strategies	21
3.4.3	Digital Signal Detrending, Normalization, and Filtering	22
3.4.4	Stratified Dataset Splitting	23
3.4.5	Signal Extraction and Processing Implementation	24
3.5	Phase 4: AI Modeling	26
4	Results	27
5	Conclusion	28
	References	29
	Appendix A:	30
.0.1	Forward & Stop	30
.0.2	Right	31
.0.3	Left	31
.0.4	center	31
.0.5	Toggle System (ON / OFF)	32
.0.6	Alarm	32

Chapter 1

Introduction

Chapter 2

Related Work

Chapter 3

Methodology

3.1 Introduction

This chapter presents the complete, end-to-end technical methodology designed and implemented in this graduation project to achieve non-invasive blood glucose level estimation. Developing a reliable health-tech system requires a strict sequence of engineering steps, moving from physical data acquisition to software optimization, signal handling, and future predictive modeling.

To maintain a rigorous scientific flow, our operational pipeline is structured into successive development phases. This chapter comprehensively details the work executed during the first major milestones of our project:

1. **Phase 1: Data Acquisition** – Building the mobile sensing client, conducting the clinical campaign, and logging ground truth values.
2. **Phase 2: Data Preprocessing and Structuring** – Lossless video orientation normalization, automated directory schema execution, and quality control filtering.

3.2 Phase 1: Data Acquisition

To train any deep learning model for estimating blood glucose non-invasively, we need a reliable dataset. This dataset must connect raw signals from a smartphone to actual, accurate blood glucose readings. Since there are no clean, publicly available datasets that

match smartphone camera readings with real medical measurements, we had to build our own data collection system.

This section explains how we designed a mobile application called *DiabetesCollector* and how our team organized a real-world data collection campaign at the University Hospital to gather data from actual patients.

3.2.1 The Data Collection Campaign at University Hospital

To make sure our project is based on real clinical data, our faculty provided us with certified medical equipment, and we practically implemented our work in a real medical environment.

Medical Equipment Provided by Faculty

Our college provided our team with official **Accu-Chek** capillary blood glucometers along with their chemical test strips and sterile lancets. The Accu-Chek device is widely trusted in hospitals and clinics for its high accuracy, making it the perfect tool to establish our reference target (the ground truth).

Hospital Environment and Ethics

We conducted our data collection campaign inside the **University Hospital**. This allowed us to interact with a wide variety of people, including healthy individuals and diabetic patients with different blood sugar levels. Before starting any measurement, we followed basic ethical steps:

- We introduced ourselves and explained the goal of our graduation project to the participant.
- We obtained verbal consent from each patient before taking any measurements.
- We followed safety protocols by disinfecting the patient's finger before using the lancet and safely disposing of the medical waste.

3.2.2 Our Android Application: DiabetesCollector

To capture the raw optical data from the patient’s finger, we developed a custom Android application named *DiabetesCollector*. We designed the app to be fast, simple, and stable so it would not crash or freeze during hospital visits.

How the App was Built

- **Programming Language & UI:** The app is written in **Kotlin** using **Jetpack Compose** to create a clean, modern user interface.
- **Architecture (Clean Architecture & UDF):** We organized the code using Clean Architecture principles. We also used Unidirectional Data Flow (UDF) through a **ViewModel**. This structure ensures that user actions and UI updates happen in a reliable sequence, preventing data loss while recording.
- **Database (ObjectBox):** Instead of using heavy SQL databases, we integrated **ObjectBox**, which is a lightweight NoSQL database. It securely saves patient details and session forms on the phone instantly.

Using the Camera (CameraX API)

The most critical part of the app is the camera controller, which uses the **Android Jetpack CameraX API**. When the recording starts, the app automatically turns on the phone’s back LED flash (**Torch Mode**). This shines a strong light through the patient’s finger tissue, allowing the camera sensor to view the blood volume changes beneath the skin (Photoplethysmography or PPG signal).

3.2.3 Patient Profiles and Ground Truth Entry

Entering Demographic Information

Before recording any signal, we asked the patient a few simple questions to build their profile in the app. These factors help the deep learning model learn how different bodies affect the signal:

- **Personal Details:** Name, Age, and Biological Gender.

- **Body Metrics:** Height (in cm) and Weight (in kg).
- **Medical History:** Smoking status (Yes/No) and whether they have other chronic health issues.

Logging the Reference Glucose Level

Right before or after recording the finger video, we checked the patient's actual blood sugar using the **Accu-Chek** glucometer. We then manually typed this exact number (in *mg/dL*) into the application's form, which is structured to capture both historical and clinical metrics seamlessly as shown in Figure 3.1. This number acts as the label or "ground truth" that our deep learning model will try to predict.

The screenshot displays a mobile application interface for entering a new record. At the top, there are two tabs: 'New Record' (active) and 'All Data'. Below the tabs, a message states 'Signal is not recorded yet' with a 'Switch to camera view' button. The form includes radio buttons for 'Gender' (Male selected) and 'Smoking' (No selected). Text input fields are provided for 'Name' (Patient1), 'Age' (34), 'Height (cm)' (180), 'Weight (kg)' (86), 'Other diseases' (None), and 'Glucose Level' (80). At the bottom, there are 'Clear' and 'Save' buttons. The interface is designed with a light orange background and a dark brown bottom navigation bar.

Figure 3.1: The New Record Form interface where patient demographics and Accu-Chek readings are entered.

Tracking Different Phone Hardware

Since our team members used different smartphones to collect data, the camera sensors were not identical. To handle this, our app automatically reads the phone's hardware properties from the Android operating system. It saves the **brand**, **model**, and **manufacturer** with every session, so we can fix any hardware differences later during data preprocessing.

3.2.4 Handling Data without Overlaps (UUID Strategy)

Since multiple team members were gathering data at the hospital using different phones at the same time, we faced a technical challenge: how do we merge all the data into one computer later without mixing up patient IDs?

If we used normal numbers like 1, 2, 3, Phone A and Phone B would both create a "Patient #1", causing data corruption during merging. To solve this, we used **Universally Unique Identifiers (UUID Version 4)**. Every time a session is created, the app generates a unique 128-bit random ID. Because this ID is globally unique, we successfully merged all databases from all phones with zero errors or overlaps.

3.2.5 The Signal Recording Process (The 20-Second Rule)

To get clean data with the highest quality, we followed a strict, step-by-step process for every patient. The participant is asked to sit comfortably and place the tip of their index finger gently over the rear camera lens and flash. They must cover the lens completely without pressing too hard, as excessive pressure blocks capillary blood flow.

The collector then verifies via the live screen preview that the entire matrix view has turned deep red, indicating proper light transmittance through the vascular tissue. Once validated, the active 20-second countdown recording session is initiated to capture between 20 to 35 complete cardiac cycles, providing a stable window for future signal analysis while preventing thermal throttling of the hardware. The user interfaces designed for handling this visual alignment and recording process are shown in Figure 3.2.

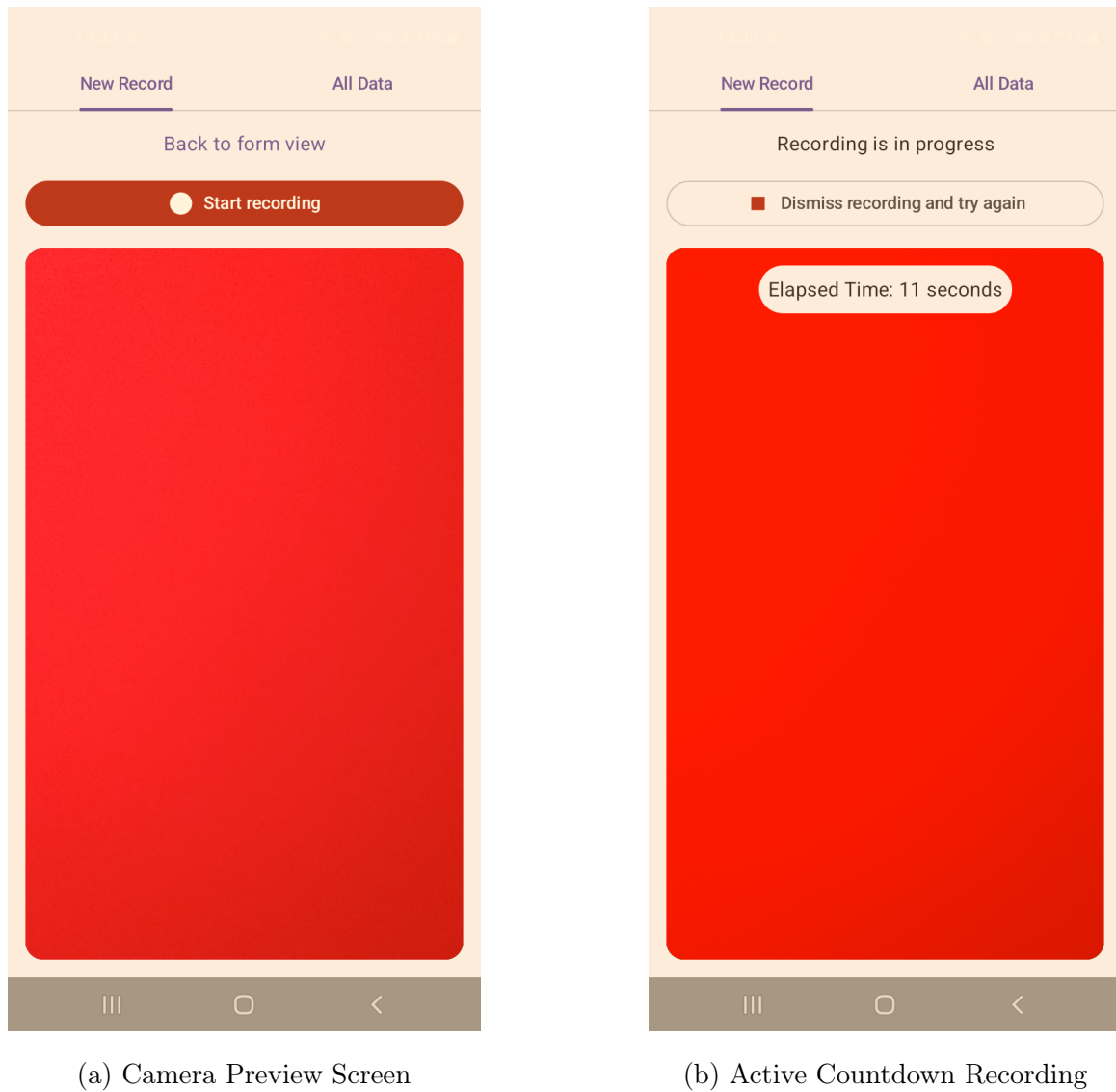


Figure 3.2: The user interfaces designed for checking the finger alignment and executing the 20-second signal capture sequence.

3.2.6 Exporting and Merging the Final Dataset

File Structure on the Phone

After completing our visits to the University Hospital, we used the built-in export feature in our app. It compresses everything into a single ZIP archive, formatted as:

`diabetes_collector_export_[DATE].zip`.

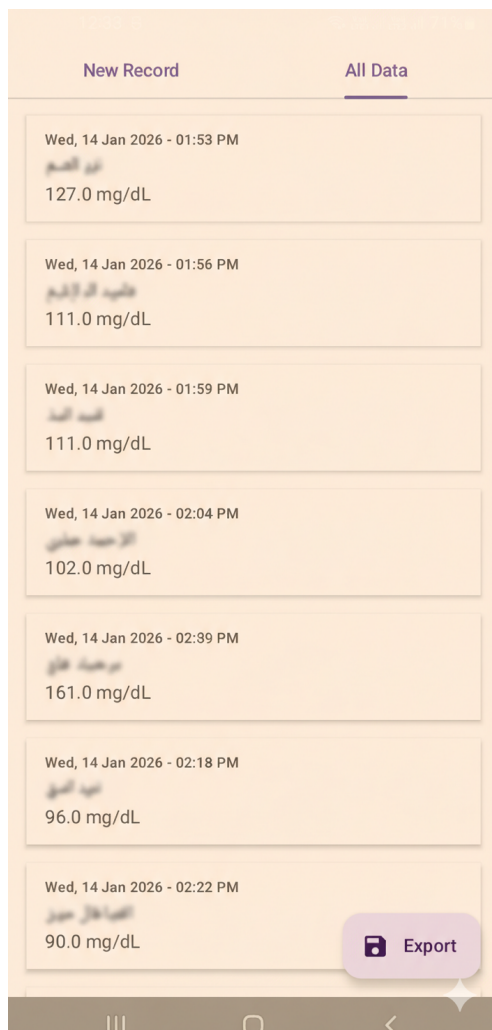
Inside the folder, the raw file components look like this:

```

diabetes_collector_export_[DATE].zip/
|
|-- patients.csv
|-- video_files/
|   |-- uuid_1.mp4
|   |-- uuid_2.mp4
|   |-- uuid_n.mp4

```

The application provides intuitive screens to manage this local storage and to trigger the internal zip compression engine, as illustrated in Figure 3.3.



(a) Local Database History View



(b) ZIP File Generation Screen

Figure 3.3: Data management screens showing local storage monitoring and the final data export engine workflow.

The Metadata CSV File

The `patients.csv` file is a big table that contains all the text and numbers we entered, linked to the video files using the unique UUIDs. Table 3.1 shows how this data is organized.

Table 3.1: The Structure of the Exported `patients.csv` File

Field Name	Data Type	Description / What it Stores
<code>uuid</code>	String	Unique ID generated for this session.
<code>name</code>	String	Name or initials of the participant.
<code>created_at</code>	Timestamp	Date and time of the recording.
<code>video_file</code>	String	The name of the matching <code>.mp4</code> video.
<code>age</code>	Integer	Age of the patient in years.
<code>gender</code>	Categorical	Male or Female.
<code>is_smoking</code>	Boolean	True if they smoke, False if not.
<code>height</code>	Float	Height of the patient in cm.
<code>weight</code>	Float	Weight of the patient in kg.
<code>other_diseases</code>	String	Any other chronic illnesses.
<code>glucose_level</code>	Float	Accu-Chek reading in mg/dL.
<code>device</code>	String	Internal Android device code.
<code>brand</code>	String	Phone brand (e.g., Samsung, Xiaomi).
<code>model</code>	String	Specific model number of the phone.
<code>manufacturer</code>	String	Company that made the phone.

Final Dataset Aggregation

At the end of the campaign, all team members moved their ZIP files to a central computer. Thanks to the UUID system, we used a short Python script to quickly combine all the CSV files and video folders into one final, large master dataset. This clean dataset is what we used to train and test our deep learning networks in the next phases of the project.

3.3 Phase 2: Data Preprocessing and Structuring

After completing the data collection campaign at the University Hospital, we succeeded in gathering a raw corpus of patient metadata and video recordings. However, raw data collected in real-world environments is rarely ready for immediate use in deep learning models.

Before feeding the signals into our neural networks, we executed a rigorous data pre-processing and structuring pipeline. This technical pipeline involved normalizing video spatial orientations, building an automated and clean directory tree, and filtering out noisy or corrupted samples to guarantee high clinical data quality.

3.3.1 Video Orientation Normalization

During the hospital campaign, team members recorded videos using different smartphones. Due to variations in how phones were held or how their built-in camera sensors handle orientation metadata, some videos were saved in landscape mode, while others were saved in portrait mode. To make the dataset consistent, we needed to make all videos follow a unified **Portrait format**.

The Challenge with Standard Video Editors

Initially, we tried using standard commercial video software and players, such as VLC Media Player and basic graphical video editors, to rotate the incorrectly oriented videos. However, we discovered a major technical drawback: these tools re-encoded the videos in a way that altered or compressed the raw pixel values. Since our project relies on extracting micro-variations in blood volume from subtle color changes in the pixels (PPG signal), any artificial alteration or compression of pixel values would corrupt our data.

The Automated FFmpeg Command-Line Solution

To solve this issue, we turned to **FFmpeg**, a powerful, lossless command-line tool for handling multimedia files. By running FFmpeg directly through a Windows Batch (`.bat`) script, we were able to process and rotate the videos directly without changing or damaging the underlying raw pixel integrity.

To automate this workflow and ensure it fits standard document margins without cutting off long parameters, we executed the multi-line structured command on the target video files as follows:

```
ffmpeg -i input.mp4 -vf "transpose=1" \  
      -c:v libx264 -crf 0 -preset veryslow \  
      -c:a copy output.mp4
```

The technical parameters of this command are carefully selected as follows:

- `-vf "transpose=1"`: This filter rotates the video exactly 90 degrees clockwise, successfully converting landscape videos into the required unified portrait format.
- `-crf 0`: Setting the Constant Rate Factor (CRF) to 0 forces the encoder into a completely **lossless mode**, guaranteeing that not a single pixel value or color detail is modified during the rotation process.
- `-preset veryslow`: This ensures the highest possible software compression efficiency and layout accuracy, preserving the strict frame-rate consistency necessary for time-series analysis.

3.3.2 Hierarchical Dataset Re-organization

When files are exported directly from our mobile application, they are packed into large folders containing unorganized data sheets and video files. To make it easier for our team and future Python deep learning scripts to browse the dataset, we decided to convert the exported directory structure into a well-defined hierarchical data tree.

The custom cleaned dataset layout was successfully organized into the following structure:

```
cleaned_data/
|
|-- data.csv
|-- patient1.mp4
|-- patient2.mp4
|-- patients_csvs/
    |-- patient1.csv
    |-- patient2.csv
```

As shown above, the main directory contains the global `data.csv` file and the raw `.mp4` recordings directly, while a dedicated subfolder named `patients_csvs/` isolated the individual telemetry files for each patient session.

Individual Patient CSV Schema Specification

For each patient session, a separate, isolated CSV file is generated and maintained within the dataset tree. This file records the primary baseline features of the patient along with placeholder columns for the signal processing arrays. Table 3.2 illustrates how the internal attributes and column headers are structured for a single patient sample.

Table 3.2: Structural Layout and Attributes of an Individual Patient CSV File

name	age	gender	glucose_level	brand	model	device	video_name	RPPG	x	y
patient1	27	Male	87	realme	RMX3636	RE5C7CL1	patient1.mp4			

The columns `RPPG`, `x`, and `y` are deliberately left empty at this operational stage. These fields represent the extracted Photoplethysmography waveform arrays and mathematical coordinate tracking matrices, which will be computationally extracted, processed, and saved during the feature engineering pipeline detailed in the subsequent phases.

3.3.3 Quality Control and Data Filtering

A critical part of preparing datasets for medical or health-tech machine learning models is strict quality control. During our initial collection campaign, we successfully registered a total of **260 samples**. However, after a rigorous auditing and inspection process, the final size of our verified clean dataset was reduced to **248 samples**.

This reduction of 12 samples was necessary due to two main real-world challenges:

1. **Signal and Video Quality Degradation:** Some 20-second video recordings contained tracking anomalies. For instance, if a patient accidentally moved their finger during the recording, or if the finger pressure was unstable, the light saturation changed dramatically, resulting in a noisy, distorted PPG waveform. These bad samples were completely discarded to protect our model from learning corrupt patterns.
2. **Human Typographical Errors (Typos):** Gathering clinical data over long, continuous shifts at the hospital is physically demanding. Due to human fatigue after hours of testing patients, minor typographical errors occurred—such as accidentally entering a wrong digit or confusing medical values during record submission.

Instead of keeping flawed records or guessing the correct data, we took a strict scientific approach and completely removed any sample that had a mismatch or a typing error. By dropping these 12 problematic entries, we ensured that our remaining 248 samples are completely clean, reliable, and highly accurate, providing a solid foundation for training the deep learning networks.

3.3.4 Automation Python Script for Restructuring

To automate the file management workflow, map the exported raw documents, and structure them into the targeted system paths described, we developed and executed a custom Python utility script. This automation program reads the master configuration matrix, tracks physical asset locations, and securely shifts file placements. The full implementation code used during this phase is presented below:

```

1  import csv
2  import os
3  import shutil
4
5  # Read the CSV file
6  csv_file = 'editor.csv'
7  base_path = r'F:\grad project cleaned\diabetes_collector_export_2026-01-22'
8
9  try:
10     with open(csv_file, 'r') as file:
11         reader = csv.reader(file)
12         for row in reader:
13             if len(row) >= 2:
14                 old_name = row[0].strip()
15                 new_name = row[1].strip()
16
17                 old_path = os.path.join(base_path, old_name)
18                 new_path = os.path.join(base_path, new_name)
19
20                 if os.path.exists(old_path):
21                     shutil.move(old_path, new_path)
22                     print(f"Renamed: {old_name} -> {new_name}")
23                 else:
24                     print(f"File not found: {old_name}")
25
26 except FileNotFoundError:
27     print(f"CSV file '{csv_file}' not found")
28 except Exception as e:
29     print(f"Error: {e}")

```

Listing 1: Python Script for Automated Dataset Hierarchy Restructuring

3.4 Phase 3: rPPG Extraction

Once the raw video database was securely organized and unified into a standard spatial layout, the core signal processing pipeline was executed. The main objective of this phase is to transform raw spatiotemporal video information into continuous, clean, one-dimensional Photoplethysmography (PPG) waveforms.

This technical phase encompasses four critical consecutive steps: raw color channel

extraction, frame rate normalization analysis, digital signal filtering, and balanced dataset splitting for future machine learning modeling.

3.4.1 Raw Temporal RGB Extraction

To observe the micro-variations of blood volume pulsing through the capillary tissue, the spatial frame sequences must be reduced to temporal tracking arrays. Instead of isolating a localized geometric Region of Interest (ROI) which may risk dropping crucial cardiac properties due to minor finger movements, a full-frame averaging strategy was implemented.

For each successive video frame, the spatial matrix containing thousands of pixels is compressed into a single, representative absolute mean value for the Red (R), Green (G), and Blue (B) channels. Mathematically, for a frame matrix F of size $M \times N$, the average channel intensity C_{avg} (where $C \in \{R, G, B\}$) is defined as:

$$C_{\text{avg}} = \frac{1}{M \times N} \sum_{i=1}^M \sum_{j=1}^N F(i, j, C) \quad (3.1)$$

Given that each patient session consists of a strict 20-second video recorded at a standardized rate of 20 frames per second, this computational reduction yields exactly 400 continuous mathematical points per channel (20 seconds multiplied by 20 FPS equals 400 points), capturing the comprehensive raw rPPG signal.

3.4.2 Frame Rate Normalization Strategies

A primary technical challenge in multi-device data campaigns is hardware sampling variance. Within our verified corpus of 248 patient videos, a clear hardware split was present: 93 videos were captured at a sampling frequency of 30 FPS, while 155 videos were recorded at 20 FPS. This discrepancy directly alters the frequency distribution during Fourier Transform calculations and destroys model training generalization.

To establish an optimal sampling benchmark, our team rigorously tested three different standardization strategies evaluated across critical metrics: Signal-to-Noise Ratio (SNR), Heart Rate (HR), and Power Spectrum density:

1. **Downsampling (30 FPS to 20 FPS):** Representing the 30 FPS signal onto a

20 FPS scale by decimation, effectively scaling the time-series length by a factor of two-thirds.

2. **Upsampling / Interpolation (20 FPS to 30 FPS):** Utilizing cubic spline interpolation algorithms to reconstruct a 20 FPS signal over a dense 30 FPS matrix by a factor of three-halves.
3. **Combination Tuning (20/30 FPS to 25 FPS):** Shifting both hardware categories into a central 25 FPS frequency via balanced up/down resampling filters.

The empirical evaluation of these methods demonstrated that the physical variations across all three temporal options were virtually negligible. Because the difference between the native recording speeds is only 10 FPS, the core physiological parameters—such as the calculated Heart Rate (HR)—remained perfectly invariant with a delta of zero. Consequently, **20 FPS was officially selected as our absolute dataset standard.** This downsampling approach preserves the original, un-interpolated biological data points while completely avoiding artificial data artifacts caused by upsampling interpolation.

3.4.3 Digital Signal Detrending, Normalization, and Filtering

Capillary blood signals extracted from mobile hardware are inherently buried under intensive high-frequency noise, respiratory baseline drifts, and high-amplitude somatic motion artifacts. To isolate the pure cardiac pulse waveform, a robust signal conditioning architecture was developed using SciPy mathematical engines. The sequential signal processing steps applied to each raw channel are as follows:

- **Linear Detrending:** Removing slow, non-physiological baseline wanders caused by gradual thermal changes or minor patient pressure variations using linear least-squares adjustments.
- **Z-Score Normalization:** Standardizing the signal amplitude to zero mean and unit variance, eliminating scale discrepancies between different smartphone camera sensors. The normalized signal x_{norm} is computed via:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \quad (3.2)$$

where μ is the signal mean and σ represents the standard deviation.

- **Butterworth Bandpass Filtering:** Passing the standardized signal through a 4th-order digital Butterworth bandpass filter (0.5 Hz to 8.0 Hz) to eliminate non-cardiac noise frequencies.

3.4.4 Stratified Dataset Splitting

To prepare our processed arrays for machine learning and deep learning regression modeling, a highly controlled data splitting strategy was mandatory. Splitting medical datasets randomly often introduces catastrophic class imbalances, where certain critical blood glucose ranges might be completely missing from the training or testing subsets.

To prevent this bias, our verified collection of 248 patient records was explicitly stratified into 5 distinct clinical categories based on the ground-truth Accu-Chek measurements. The breakdown of our data distribution across these glycemic tiers is detailed in Table 3.3.

Table 3.3: Distribution of Patient Records Across Glycemic Tiers

Glucose Level Category	Glycemic Range (mg/dL)	Number of Samples
Tier 1: Normal / Low	Less than 120	87
Tier 2: Pre-diabetic	Between 120 and 200	101
Tier 3: Diabetic Hyper	Between 200 and 300	34
Tier 4: Critical Hyper	Between 300 and 400	15
Tier 5: Extreme Emergency	Greater than 400	11
Total Corpus	-	248 Samples

A strict Stratified Partitioning Rule of 70% Training, 15% Validation, and 15% Testing was applied uniformly across each of the 5 individual categories. By extracting these identical proportions from each specific tier, we guaranteed that the final structured folders maintain an exact, mirrored representation of the overall dataset distribution. The final operational division yielded 170 samples for the Train subset, 38 samples for the Validation subset, and 40 samples for the Unseen Testing subset, creating an optimal, un-biased foundation for model evaluation.

3.4.5 Signal Extraction and Processing Implementation

To enforce reproducibility and automate this massive signal extraction matrix, a custom Python utility was written using OpenCV (`cv2`) and SciPy Signal libraries. The production script automatically ingests the normalized videos, performs spatial frame reduction, normalizes sampling rates, executes the 4th-order Butterworth bandpass filter, and exports structural sequential database tables.

The complete processing script implemented during this stage is locked in its precise structural position and documented in Listing 2, allowing the code text to split and flow naturally across sequential page boundaries without shifting past Phase 4:

Listing 2: Comprehensive Python Engine for Full-Frame rPPG Extraction and Signal Conditioning

```

1  import cv2
2  import numpy as np
3  import pandas as pd
4  import os
5  from scipy.signal import resample, butter, filtfilt, detrend
6
7  # Configuration Paths
8  video_folder = r"G:\cleaned data"
9  output_folder = r"C:\Users\Mostafa Magdy\Desktop\processed_dataset"
10 data_csv_path = r"G:\cleaned data\data.csv"
11
12 os.makedirs(output_folder, exist_ok=True)
13 df = pd.read_csv(data_csv_path)
14
15 # Filter Parameters
16 fs = 20
17 lowcut = 0.5
18 highcut = 8.0
19
20 def extract_raw_rgb(video_path):
21     cap = cv2.VideoCapture(video_path)
22     R, G, B = [], [], []
23     while True:
24         ret, frame = cap.read()
25         if not ret:
26             break

```

```

27         # Mean of full frame matrix (ROI = Full Frame)
28         R.append(np.mean(frame[:, :, 2]))
29         G.append(np.mean(frame[:, :, 1]))
30         B.append(np.mean(frame[:, :, 0]))
31     cap.release()
32     return np.array(R), np.array(G), np.array(B)
33
34 def downsample_30_to_20(signal):
35     target_length = int(len(signal) * 20 / 30)
36     return resample(signal, target_length)
37
38 def signal_conditioning(signal, fs, low, high, order=4):
39     # Detrend & Z-score Normalization
40     detrended = detrend(signal)
41     std = np.std(detrended)
42     normalized = detrended if std == 0 else (detrended - np.mean(detrended)) /
43     ↪ std
44
45     # Butterworth Bandpass Filter
46     nyq = 0.5 * fs
47     b, a = butter(order, [low/nyq, high/nyq], btype='band')
48     return filtfilt(b, a, normalized)
49
50 # Execution Loop over the verified dataset
51 for i in range(1, 249):
52     video_name = f"patient{i}.mp4"
53     video_path = os.path.join(video_folder, video_name)
54     if not os.path.exists(video_path):
55         continue
56
57     # Extract & Standardize Frame Rate
58     R, G, B = extract_raw_rgb(video_path)
59     if i <= 93: # Native 30fps hardware downsampled to 20fps
60         R, G, B = downsample_30_to_20(R), downsample_30_to_20(G),
61         ↪ downsample_30_to_20(B)
62
63     # Force strict array synchronization (Length = 400)
64     target_len = 400
65     if len(R) != target_len:
66         R, G, B = resample(R, target_len), resample(G, target_len), resample(B,
67         ↪ target_len)
68
69     # Apply Signal Detrending, Scaling, and Butterworth Bandpass Filter
70     R_processed = signal_conditioning(R, fs, lowcut, highcut)
71     G_processed = signal_conditioning(G, fs, lowcut, highcut)

```

```
69     B_processed = signal_conditioning(B, fs, lowcut, highcut)
70
71     # Export Stratified Tracking Sheets
72     patient_row = df[df["id"] == i].iloc[0]
73     output_df = pd.DataFrame({
74         "id": [patient_row["id"]] * target_len,
75         "name": [patient_row["name"]] * target_len,
76         "age": [patient_row["age"]] * target_len,
77         "gender": [patient_row["gender"]] * target_len,
78         "R_value": R_processed,
79         "G_value": G_processed,
80         "B_value": B_processed,
81         "glucose_level": [patient_row["glucose_level"]] * target_len
82     })
83     output_df.to_csv(os.path.join(output_folder, f"patient{i}.csv"),
84                     ↪ index=False)
85
86 print("Phase 3 Signal Extraction completed successfully!")
```

3.5 Phase 4: AI Modeling

This section will detail the neural network architectures, hyperparameter tuning, and regression metrics applied to map the processed signals into clinical glucose values.

Chapter 4

Results

Chapter 5

Conclusion

References

Appendix A:

A.1: Face Movement

.0.1 Forward & Stop

```
if (isMoving):
    if (pitch_direction == "d" and not(stopped)):
        USART_SEND('S')
        stopped = True
    if (pitch_direction == "c" and stopped):
        isMoving = False
        stopped = False
        alarm_time = 0
else:
    if (pitch_direction == "d"):
        if (time_start_down == 0 and face_exist):
            time_start_down = time.time()
        if (time.time() - time_start_down > 2 and time.time() - time_start_down < 2.5):
            GPIO.output(BUZZER_PIN, GPIO.HIGH)
    else:
        GPIO.output(BUZZER_PIN, GPIO.LOW)
    if (time_start_down and pitch_direction == "c"):
        if (time.time() - time_start_down > 2 and face_exist):
            isMoving = True
            USART_SEND('F')
            time_start_down = 0
```

```
alarm_time = 0
```

.0.2 Right

```
if (right):
    GPIO.output(RIGHT_PIN, GPIO.HIGH)
else:
    GPIO.output(RIGHT_PIN, GPIO.LOW)
if (yaw_direction == "r" and pitch_direction == "c" and not(right)):
    right = True
    GPIO.output(RIGHT_PIN, GPIO.HIGH)
    USART_SEND('R')
if (yaw_direction == "c"):
    right = False
```

.0.3 Left

```
if (yaw_direction == "l" and pitch_direction == "c" and not(left)):
    left = True
    USART_SEND('L')
if (yaw_direction == "c"):
    left = False
if (right or left):
    center = True
```

.0.4 center

```
if (center and not(right) and not(left)):
    USART_SEND('C')
    center = False
```


.0.5 Toggle System (ON / OFF)

```
if (pitch_direction == "u"):
    if (time_start_up == 0):
        time_start_up = time.time()

if (time_start_up and pitch_direction == "c"):
    if (time.time() - time_start_up > 2):
        USART_SEND('T')
        print("Toggled detection")
        isMoving = False
        alarm_happened = False
        GPIO.output(BUZZER_PIN, GPIO.LOW)
        can_detect = not(can_detect)
    time_start_up = 0
```

.0.6 Alarm

```
if (pitch_direction == "d" and alarm_time == 0 and face_exist):
    alarm_time = time.time()

if (alarm_time and time.time() - alarm_time >= alarm_duration):
    USART_SEND('A')
    alarm_happened = True
    can_detect = False
    time_start_down = 0
    alarm_time = 0
```